

数数量，PEFT 显著降低了微调模型的存储空间，特别适用于内存受限的设备；3) **适应性强**：PEFT 能够快速适应不同任务，而无需重新训练整个模型，使得模型在面对变化环境时具有更高的灵活性。

下面我们将通过一个具体案例来深入探讨 PEFT 技术如何显著提升参数效率。具体来说，表 4.1¹ 详细展示了在配备 80GB 显存的 A100 GPU 以及 64GB 以上 CPU 内存的高性能硬件环境下，对 bigscience 模型进行全量微调与采用参数高效微调方法 LoRA（该方法将在 4.4 节中详细介绍）时，GPU 内存的消耗情况对比。根据该

表 4.1: 全量参数微调 and 参数高效微调显存占用对比（OOM 代表超出内存限制）。

模型名	全量参数微调	参数高效微调 (LoRA)
bigscience/T0_3B	47.14GB GPU / 2.96GB CPU	14.4GB GPU / 2.96GB CPU
bigscience/mt0-xxl (12B params)	OOM GPU	56GB GPU / 3GB CPU
bigscience/bloomz-7b1 (7B params)	OOM GPU	32GB GPU / 3.8GB CPU

表格可以看出，对于 80GB 显存大小的 GPU，全量参数微调 7B/12B 参数的模型，会导致显存直接溢出。而在使用 LoRA 后，显存占用被大幅缩减，使得在单卡上微调大语言模型变得可行。

本节首先介绍了对大语言模型进行下游任务适配的两类主流范式：上下文学习和指令微调。然而，由于性能和计算成本方面的限制，这两类范式难以适应需求。因此，需要研究参数高效微调技术，即 PEFT。PEFT 仅对模型的一小部分参数进行更新，在保证不牺牲性能的前提下有效减少了模型微调所需的参数量，从而节约了计算和存储资源。本节我们对主流 PEFT 方法进行了分类，在后续小节中，我们将根据本节给出的分类学详细介绍主流的三类 PEFT 方法：参数附加方法 4.2、参数选择方法 4.3 以及低秩适配方法 4.4，并探讨 PEFT 的相关应用与实践 4.5。

¹表格数据来源：<https://github.com/huggingface/peft>

4.2 参数附加方法

参数附加方法 (Additional Parameter Methods) 通过增加并训练新的附加参数或模块对大语言模型进行微调。参数附加方法按照附加位置可以分为三类：加在输入、加在模型以及加在输出。本节将对三类方法的各自代表性工作进行具体介绍。

4.2.1 加在输入

加在输入的方法将额外参数附加到模型的输入嵌入 (Embedding) 中，其中最经典的方法是 Prompt-tuning [23]。Prompt-tuning 在模型的输入中引入可微分的连续张量，通常也被称为软提示 (Soft prompt)。软提示作为输入的一部分，与实际的文本数据一起被送入模型。在微调过程中，仅软提示的参数会被更新，其他参数保持不变，因此能达到参数高效微调的目的。

具体地，给定一个包含 n 个 token 的输入文本序列 $\{w_1, w_2, \dots, w_n\}$ ，首先通过嵌入层将其转化为输入嵌入矩阵 $X \in \mathbb{R}^{n \times d}$ ，其中 d 是嵌入空间的维度。新加入的软提示参数被表示为软提示嵌入矩阵 $P \in \mathbb{R}^{m \times d}$ ，其中 m 是软提示长度。然后，将软提示嵌入拼接上输入嵌入矩阵，形成一个新矩阵 $[P; X] \in \mathbb{R}^{(m+n) \times d}$ ，最后输入 Transformer 模型。通过反向传播最大化输出概率似然进行模型训练。训练过程仅软提示参数 P 被更新。图 4.3 给出了 Prompt-tuning 的示意图。

在实际使用中，软提示的长度范围是 1 到 200，并且通常在 20 以上就能有一定的性能保证。此外，软提示的初始化对最终的性能也会有影响，使用词表中的 token 或者在分类任务中使用类名进行初始化会优于随机初始化。值得一提的是，Prompt-tuning 原本被提出的动机不是为了实现参数高效微调，而是自动学习提示词。在第三章我们提到，利用大语言模型的常见方式是通过提示工程，也被称为硬提示 (Hard prompt)，这是因为我们使用的离散 prompt 是不可微分的。问题在于

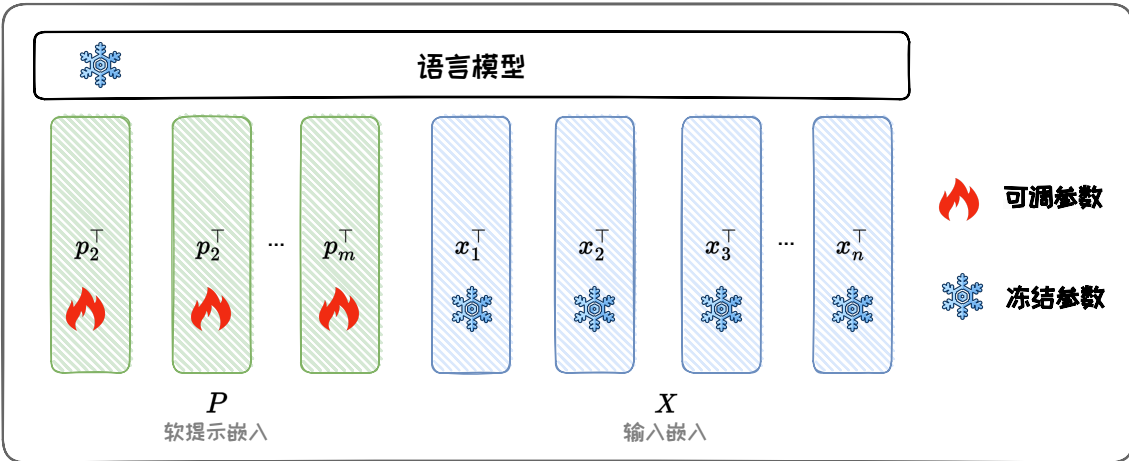


图 4.3: Prompt-tuning 示意图。

大语言模型的输出质量高度依赖于提示词的构建，要找到正确的“咒语”使得我们的大语言模型表现最佳，需要花费大量时间。因此，采用可微分的方式，通过反向传播自动优化提示词成为一种有效的方法。

总的来说，Prompt-tuning 有以下优势：(1) **内存效率高**：Prompt-tuning 显著降低了内存需求。例如，T5-XXL 模型对于特定任务的模型需要 11B 参数，但经过 Prompt-tuning 的模型只需要 20480 个参数（假设软提示长度为 5）；(2) **多任务能力**：可以使用单一冻结模型进行多任务适应。传统的模型微调需要为每个下游任务学习并保存任务特定的完整预训练模型副本，并且推理必须在单独的批次中执行。Prompt-tuning 只需要为每个任务存储一个特定的小的任务提示模块，并且可以使用原始预训练模型进行混合任务推理（在每个任务提示词前加上学到的 soft prompt）。(3) **缩放特性**：随着模型参数数量的增加，Prompt-tuning 的性能会逐渐增强，并且在 10B 参数量下的性能接近（多任务）全参数微调的性能。

4.2.2 加在模型

加在模型的方法将额外的参数或模型添加到预训练模型的隐藏层中，其中经典的方法有 Prefix-tuning [24]、Adapter-tuning [18] 和 AdapterFusion [35]。

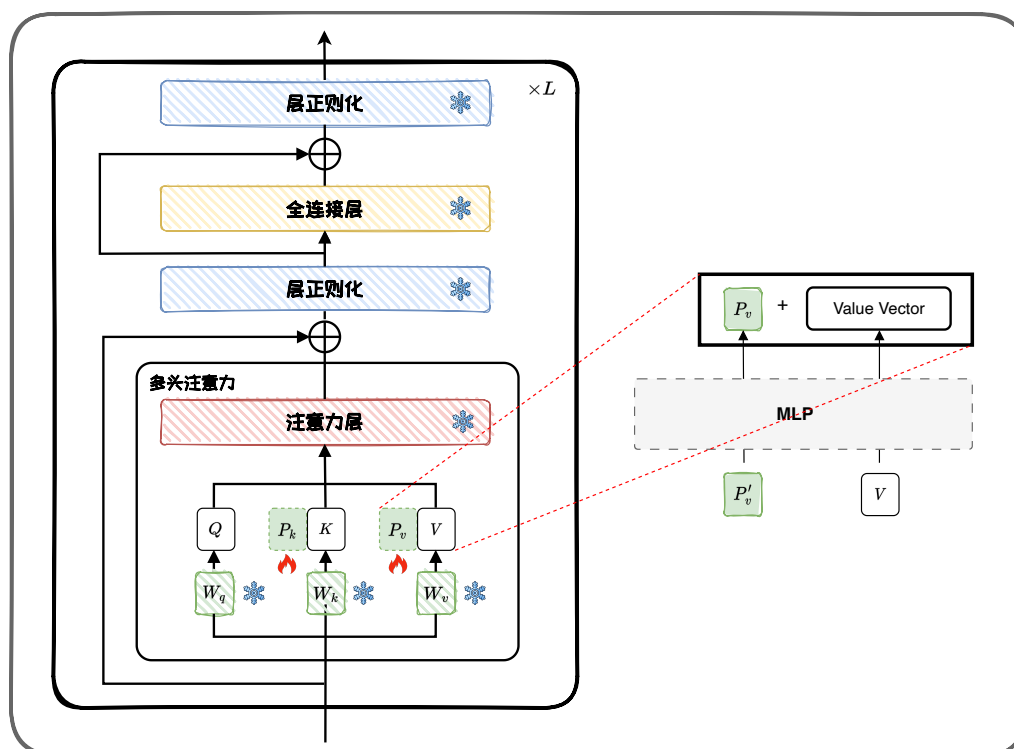


图 4.4: Prefix-tuning 示意图。

1. Prefix-tuning

Prefix-tuning 和上一节介绍的 Prompt-tuning 十分类似，区别在于 Prompt-tuning 仅将软提示添加到输入嵌入中，而 Prefix-tuning 将一系列连续的可训练前缀（Prefixes，即 Soft-prompt）插入到输入嵌入以及 Transformer 注意力模块中，如图 4.4 所示， P_k 和 P_v 是插入到 Transformer block 中的前缀。相比 Prompt-tuning，Prefix-tuning 大幅增加了可学习的参数量。

具体而言，Prefix-tuning 引入了一组可学习的向量 P_k 和 P_v ，这些向量被添加到所有 Transformer 注意力模块中的键 K 和值 V 之前。类似于 Prompt-tuning，Prefix-tuning 也会面临前缀参数更新不稳定的问题，从而导致优化过程难以收敛。因此，在实际应用中，通常需要在输入 Transformer 模型前，先通过一个多层感知机（MLP）进行重参数化。这意味着需要训练的参数包括 MLP 和前缀矩阵两部分。训练完成后，MLP 的参数会被丢弃，仅保留前缀参数。总的来说，Prefix-tuning 有

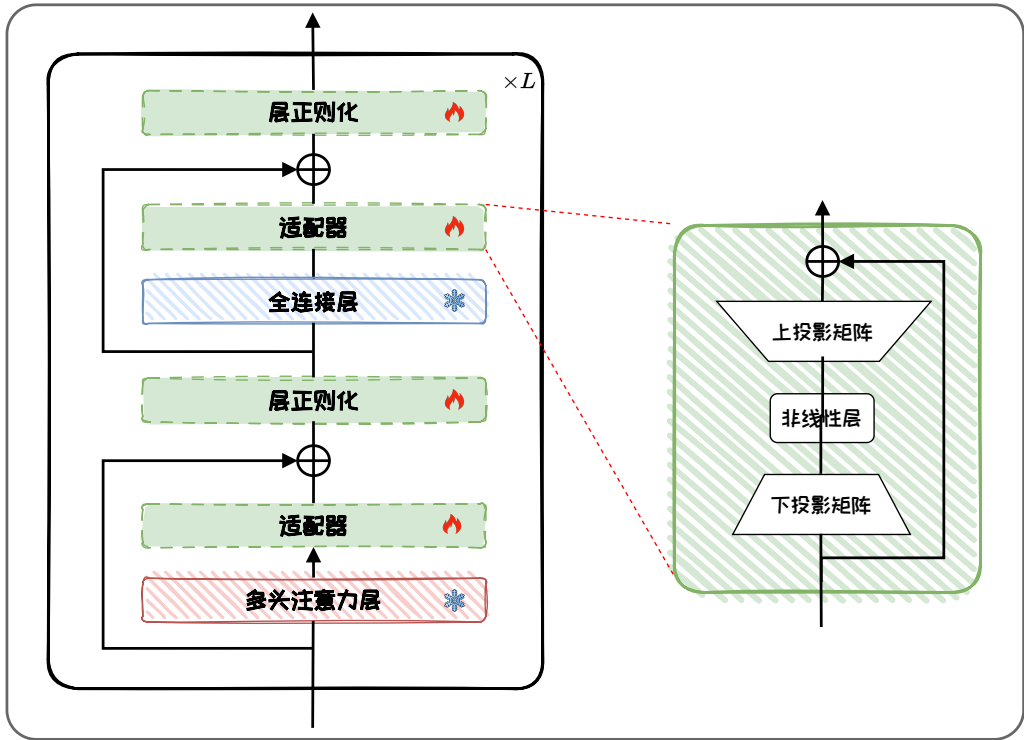


图 4.5: Adapter-tuning 示意图。

以下优势：(1) **参数效率**：只有前缀参数在微调过程中被更新，这显著减少了训练参数量；(2) **任务适应性**：前缀参数可以针对不同的下游任务进行定制，微调方式灵活；(3) **保持预训练知识**：由于预训练模型的原始参数保持不变，Prefix-tuning 能够保留预训练过程中学到的知识。

2. Adapter-tuning

Adapter-tuning [18] 向预训练语言模型中插入新的可学习的神经网络模块，称为适配器 (Adapter)。适配器模块通常采用瓶颈 (Bottleneck) 结构，即一个上投影层、一个非线性映射和一个下投影层组成的全连接模块。其中，下投影层将信息压缩到一个低维的表示，经过非线性映射后再通过上投影层扩展回原始维度。如图 4.5 所示，Adapter-tuning 在 Transformer 的每一个多头注意力层 (Multi-head Attention Layer，图中红色块) 和全连接层 (Feed-forward Network Layer，图中蓝色块) 之后添加适配器。与 Transformer 的全连接层不同，由于采用了瓶颈结构，适

适配器的隐藏维度通常比输入维度小。和其他 PEFT 方法类似,在训练时,通过固定原始模型参数,仅对适配器、层正则化(图中绿色框)以及最后的分类层参数(图中未标注)进行微调,可以大幅缩减微调参数量和计算量,从而实现参数高效微调。适配器模块的具体结构如图 4.5 右边所示,适配器模块通常由一个下投影矩阵 $W_d \in \mathbb{R}^{d \times r}$ 和一个上投影矩阵 $W_u \in \mathbb{R}^{r \times d}$ 以及残差连接组成,其中 $r \ll d$:

$$A^{(l)} = \sigma(W_d * H^{(l-1)})W_u + H^{(l-1)}, \quad (4.1)$$

其中, $\sigma(\cdot)$ 是激活函数,如 ReLU 或 Sigmoid。 $A^{(l)}$ 是适配器的输出, $H^{(l-1)}$ 是第 $l-1$ 层的隐藏状态。

在适配器中,下投影矩阵将输入的 d 维特征压缩到低维 r ,再用上投影矩阵投影回 d 维。因此,每一层中的总参数量为 $2dr + d + r$,其中包括投影矩阵及其偏置项参数。通过设置 $r \ll d$,可以大幅限制每个任务所需的参数量。进一步地,适配器模块的结构还可以被设计得更为复杂,例如使用多个投影层,或使用不同的激活函数和参数初始化策略。此外,Adapter-tuning 还有许多变体,例如通过调整适配器模块的位置 [14, 56]、剪枝 [15] 等策略来减少可训练参数的数量。

3. AdapterFusion

由于 Adapter-tuning 无需更新预训练模型,而是通过适配器参数来学习单个任务,每个适配器参数都保存了解决该任务所需的知识。因此,如果想要结合多个任务的知识,可以考虑将多个任务的适配器参数结合在一起。基于该思路,AdapterFusion 提出一种两阶段学习的方法,先学习多个任务,对每个任务进行知识提取;再“融合”(Fusion)来自多个任务的知识。具体的两阶段步骤如下:

第一阶段:知识提取。给定 N 个任务,首先对每个任务分别训练适配器模块,用于学习特定任务的知识。该阶段有两种训练方式,分别如下:

- **Single-Task Adapters(ST-A):** 对于 N 个任务,模型都分别独立进行优化,各个任务之间互不干扰,互不影响。

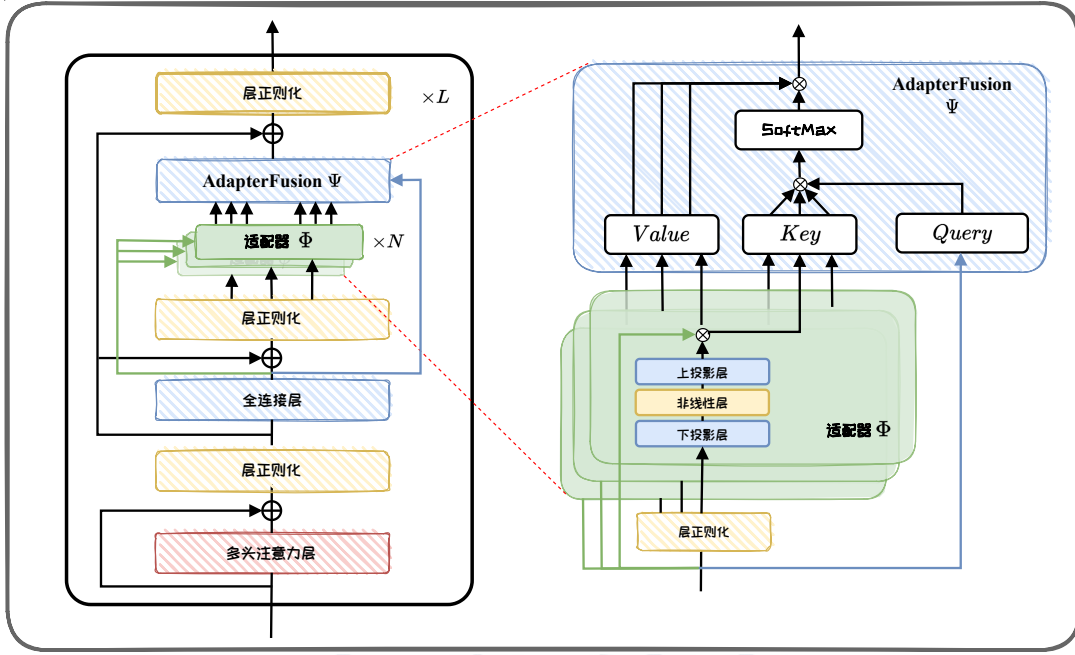


图 4.6: AdapterFusion 示意图。

- **Multi-Task Adapters(MT-A)**: 通过多任务学习对 N 个任务进行联合优化。

第二阶段：知识组合。 单个任务的适配器模块训练完成后，AdapterFusion 将不同适配器模块进行融合（Fusion），以实现知识组合。该阶段引入一个新的融合模块，该模块旨在搜索多个任务适配器模块的最优组合，实现任务泛化。在该阶段，语言模型的参数以及 N 个适配器的参数被固定，仅微调 AdapterFusion 模块的参数，并优化以下损失：

$$\Psi \leftarrow \operatorname{argmin}_{\Psi} L_n(D_n; \Theta, \Phi_1, \dots, \Phi_N, \Psi)。$$
 (4.2)

其中， D_n 是第 n 个任务的训练数据， L_n 是第 n 个任务的损失函数， Θ 表示预训练模型参数， Φ_i 表示第 i 个任务的适配器模块参数， Ψ 是融合模块参数。图 4.6 给出 AdapterFusion 的示意图。每层的 AdapterFusion 模块包括可学习的 Key（键）、Value（值）和 Query（查询）等对应的投影矩阵。全连接层的输出当作 Query，适配器的输出当作 Key 和 Value，计算注意力得到联合多个适配器的输出结果。此外，在不同的适配器模块间参数共享融合模块，可以进一步减少参数数量。

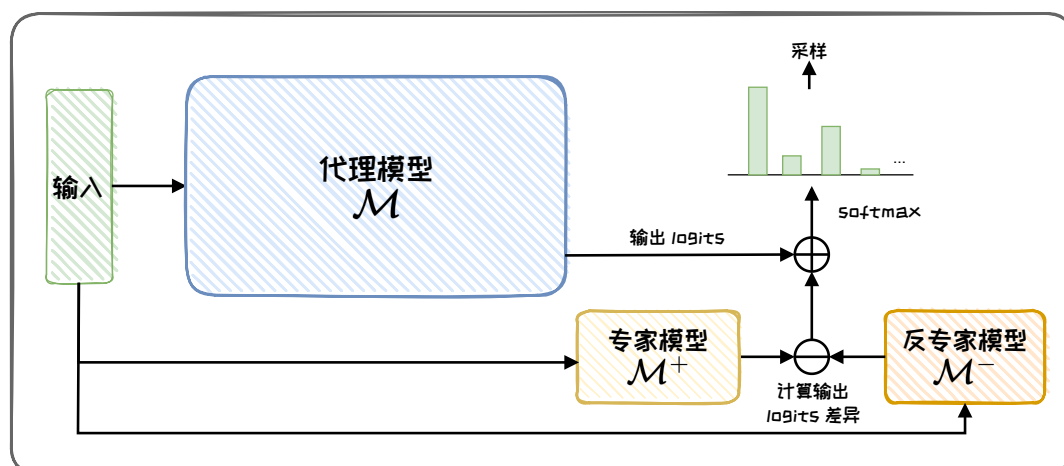


图 4.7: Proxy-tuning 示意图。

4.2.3 加在输出

在微调大语言模型时，通常会面临以下问题：首先，大语言模型的**参数数量**可能会非常庞大，例如 LLaMA 系列最大的模型拥有 70B 参数，即使采用了 PEFT 技术，也难以在普通消费级 GPU 上完成下游任务适应；其次，用户可能**无法直接访问大语言模型的权重**（黑盒模型），这为微调设置了障碍。

为了应对这些问题，代理微调（Proxy-tuning）[27] 提供了一种轻量级的解码时（Decoding-time）算法，允许我们在不直接修改大语言模型权重的前提下，通过仅访问模型输出词汇表预测分布，来实现对大语言模型的进一步定制化调整。

如图 4.7 所示，给定待微调的**代理模型 $\mathcal{M}$** 以及较小的**反专家模型**（Anti-expert model） $\mathcal{M}^-$ ，这两个模型需要相同的词汇表。我们对 $\mathcal{M}^-$ 进行微调，得到微调后的**专家模型**（Expert model） $\mathcal{M}^+$ 。在每一个自回归生成的时间步中，代理微调首先计算专家模型 $\mathcal{M}^+$ 和反专家模型 $\mathcal{M}^-$ 之间的 *logits* 分布差异，然后将其加到代理模型 $\mathcal{M}$ 下一个词预测的 *logits* 分布中。具体来说，在代理微调的计算阶段，针对每一时间步 t 的输入序列 $x_{<t}$ ，从代理模型 $\mathcal{M}$ 、专家模型 $\mathcal{M}^+$ 和反专家模型 $\mathcal{M}^-$